

Lab Sheet 09

Registration No. : 2025/CS/79
Index No. : 25000799

(01)

a. Brute Force

b.

i. Total routes = $8! = 40,320$

ii. Execution time = $40,320 / 2,000,000 = 0.02016$ Seconds (20.16 ms)

c.

Advantages =

Guarantees the optimal path

Simple to implement

Disadvantages =

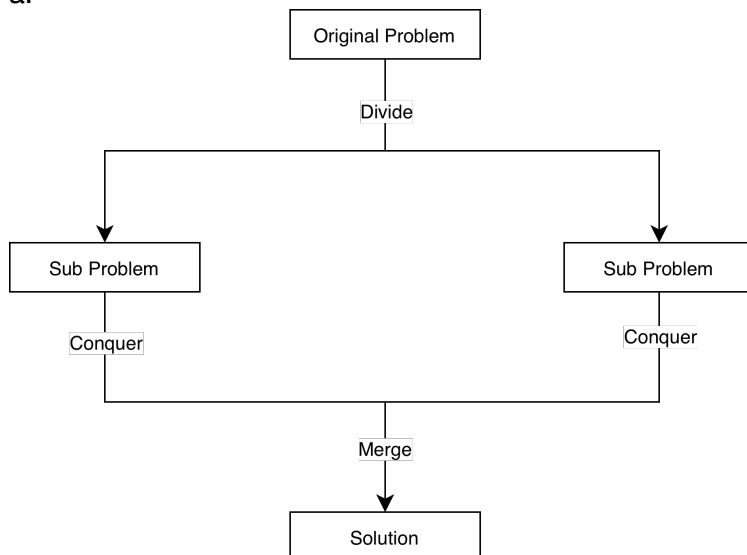
Time complexity of $O(n!)$

Wastes computational power

d. Dynamic programming - it ensures the optimal path is found using the previously calculated data instead of having to perform same calculations over and over which reduces the time complexity of $O(n!)$ and it saves the computational power wasted on those calculations.

(02)

a.



The bigger problems get divided in to smaller individual parts. Those individual parts are solved separately and merged back together to get the final solution for the original problem.

b.

c.

01 : If the array contains a single element it is already sorted.

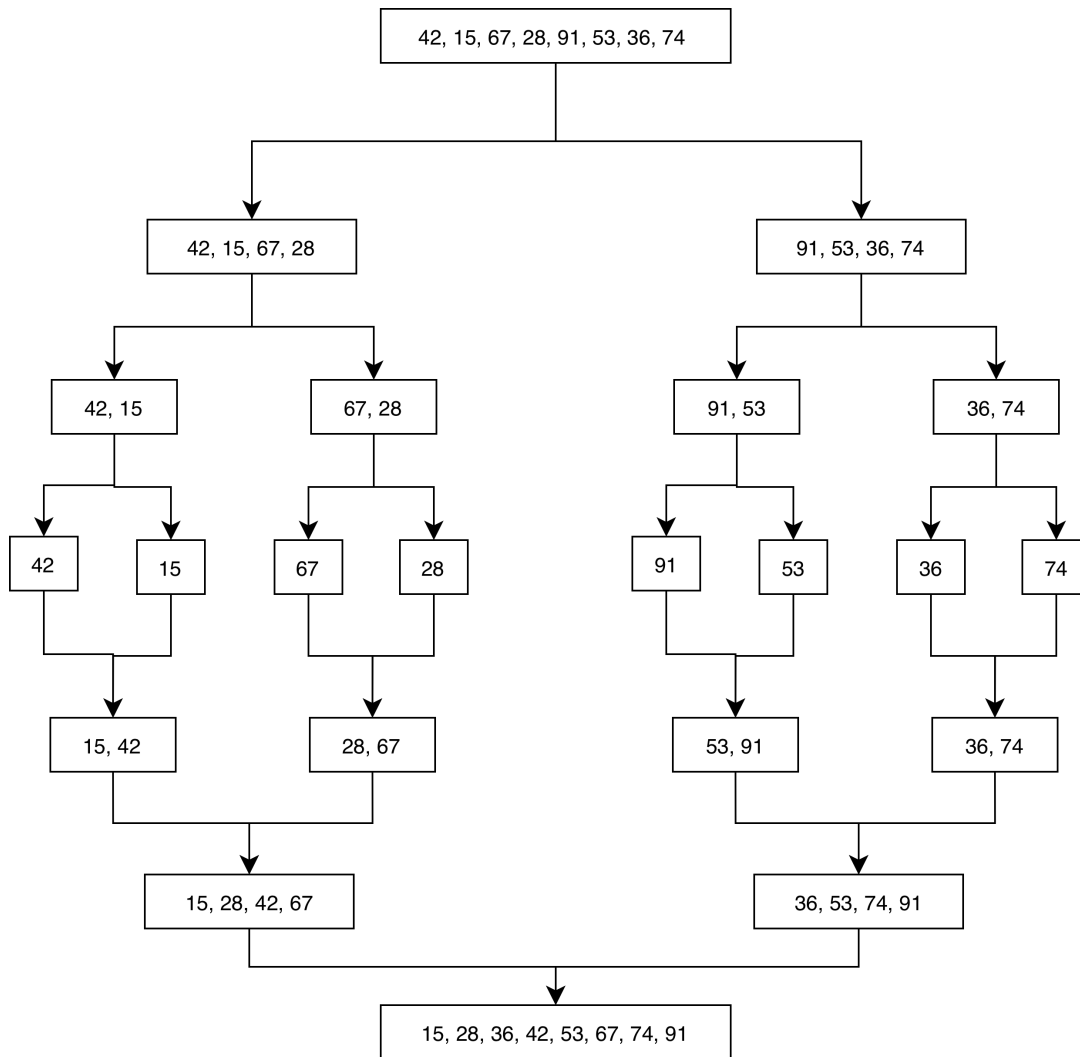
02 : Return the array.

03 : If not divide the array into two halves.

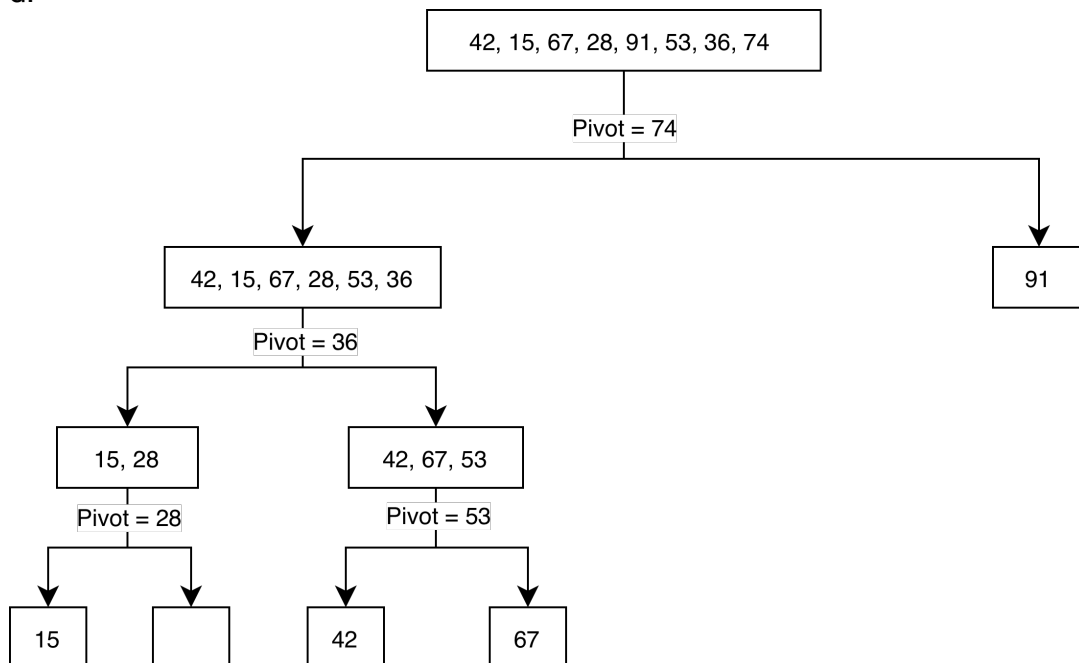
04 : Input the left half and the right half to the step 01 again.

05 : Merge the two halves back together.

06 : Final sorted array



d.



- e.
- No of levels = $\log(8) + 1 = 4$ levels
 - No of subarrays =
 - No of Merge operations = $n - 1 = 8 - 1 = 7$ operations

(03)

a.

$$A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 2 \\ 6 & 1 \end{bmatrix}$$

$$A \cdot B = \begin{bmatrix} 2*5 + 3*6 & 2*2 + 3*1 \\ 1*5 + 4*6 & 1*2 + 4*1 \end{bmatrix}$$

$$A \cdot B = \begin{bmatrix} 28 & 7 \\ 29 & 6 \end{bmatrix}$$

- b. Strassen's algorithm reduces the 8 multiplications required to calculate the multiplication of 2 by 2 matrix into 7 multiplications and set of addition and subtractions.

c.

$$\begin{aligned} P1 &= (A11 + A22)(B11 + B22) & C11 &= P1 + P4 - P5 + P7 \\ P2 &= (A21 + A22) B11 & C12 &= P3 + P5 \\ P3 &= A11 (B12 - B22) & C21 &= P2 + P4 \\ P4 &= A22 (B21 - B11) & C22 &= P1 + P3 - P2 + P6 \\ P5 &= (A11 + A12) B22 \\ P6 &= (A21 - A11)(B11 + B12) \\ P7 &= (A12 - A22)(B21 + B22) \end{aligned}$$

$$P1 = (2 + 4)(5 + 1) = 36$$

$$C11 = 36 + 4 - 5 - 7 = 28$$

$$C = \begin{bmatrix} 28 & 7 \\ 29 & 6 \end{bmatrix}$$

$$P2 = (1 + 4) * 5 = 25$$

$$C12 = 2 + 5 = 7$$

$$P3 = 2 * (2 - 1) = 2$$

$$C21 = 25 + 4 = 29$$

$$P4 = 4 * (6 - 5) = 4$$

$$C22 = 36 + 2 - 25 - 7 = 6$$

$$P5 = (2 + 3) * 1 = 5$$

$$P6 = (1 - 2)(5 + 2) = -7$$

$$P7 = (3 - 4)(6 + 1) = -7$$

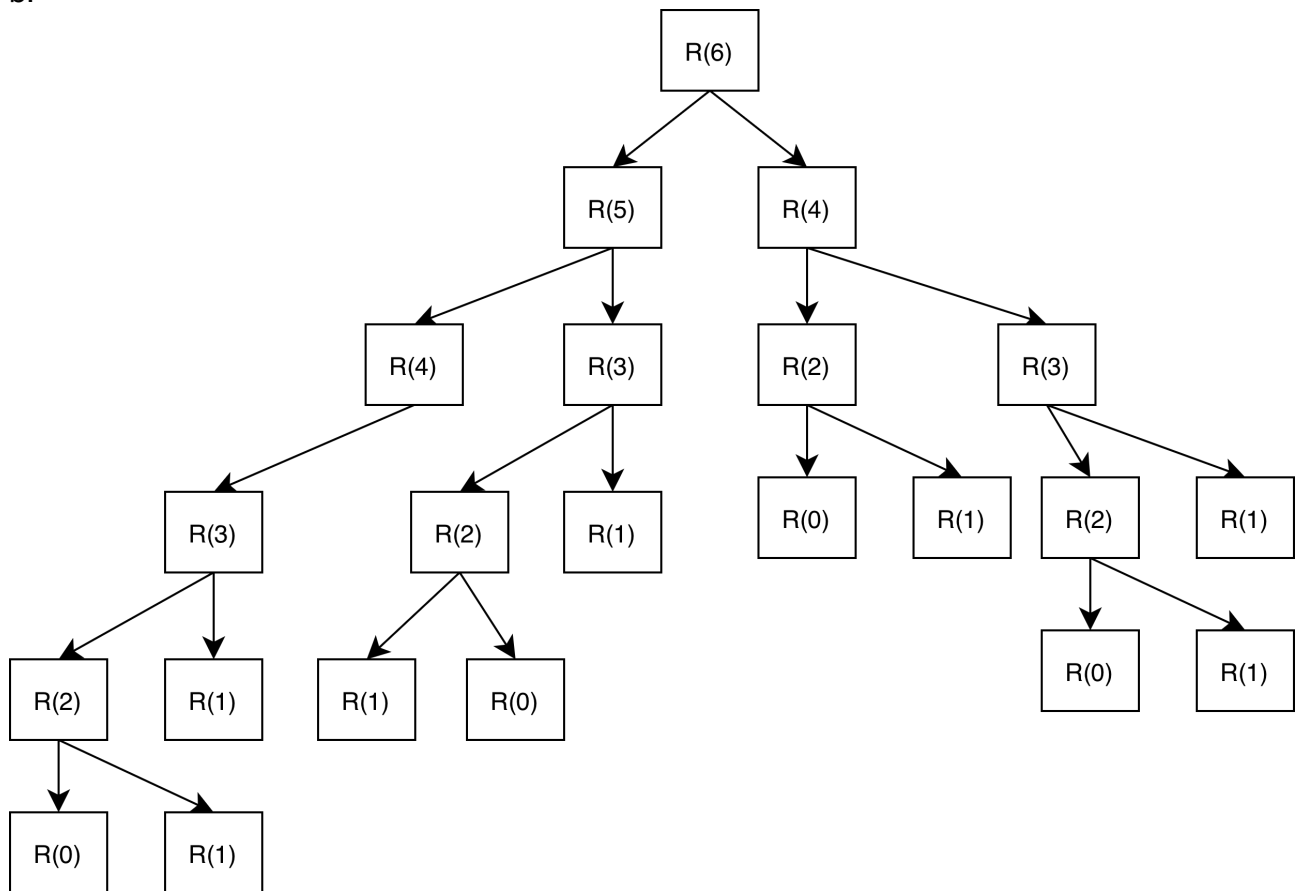
d.

	Conventional	Strassen's
Divide and Conquer	Do no use	Use to break the matrix down to 2x2 matrices
No of multiplications	8	7
Time Complexity	$O(n^3)$	$O(n^{\log 7})$
Practical advantage and limitations	Simple and numerically stable	Asymptotically faster

(04)

- a. In divide and conquer the divided smaller problems may be overlapping with other problems. In this case divide and conquer approach will perform same calculations over and over again while dynamic programming approach prevents this problem by using previously calculated data for those subproblems. This makes the dynamic programming approach more suitable for this problem.

b.



c.

[R(1)]
[R(1), R(2)]
[R(1), R(2), R(3)]
[R(1), R(2), R(3), R(4)]
[R(1), R(2), R(3), R(4), R(5)]
[R(1), R(2), R(3), R(4), R(5), R(6)]
[R(1), R(2), R(3), R(4), R(5), R(6), R(7)]
[R(1), R(2), R(3), R(4), R(5), R(6), R(7), R(8)]

d.

[R(1)]
[R(1), R(2)]
[R(1), R(2), R(3)]
[R(1), R(2), R(3), R(4)]
[R(1), R(2), R(3), R(4), R(5)]
[R(1), R(2), R(3), R(4), R(5), R(6)]
[R(1), R(2), R(3), R(4), R(5), R(6), R(7)]
[R(1), R(2), R(3), R(4), R(5), R(6), R(7), R(8)]